

KUBERNETES

My Guide to Container Orchestration

1. Introduction to Kubernetes

1.1 What Are Kubernetes?

Kubernetes (often abbreviated K8s) is an open-source platform for automating the deployment, scaling, and management of containerized applications. Originally designed by Google and now maintained by the Cloud Native Computing Foundation (CNCF), Kubernetes has become the industry standard for running applications reliably at scale.

At its core, Kubernetes takes a declarative approach: you describe the desired state of your application — how many copies should run, what resources they need, how they should be networked — and Kubernetes continuously works to make reality match that description.

1.2 Why Kubernetes Exists

Before container orchestration, teams manually managed servers, deployed applications by hand, and scaled infrastructure reactively. As applications moved to containers (via Docker and similar tools), a new problem emerged: how do you run hundreds or thousands of containers reliably across many machines?

- Automatically restart containers that fail
- Replace and reschedule containers when nodes die
- Scale applications up or down based on demand
- Roll out updates without downtime, and roll them back if something breaks
- Distribute network traffic (load balancing) across running instances
- Manage secrets, configuration, and storage consistently

1.3 Kubernetes vs. Plain Docker

Aspect	Difference
Scope	Docker runs a single container on a single host. Kubernetes orchestrates many containers across many hosts.
Scaling	Docker Compose can start multiple containers, but has no automatic scaling or self-healing.
Self-healing	Kubernetes restarts failed containers and reschedules them on healthy nodes automatically.

Aspect	Difference
Networking	Kubernetes provides built-in service discovery, load balancing, and a flat cluster network.
Rollouts	Kubernetes supports rolling updates, canary releases, and automated rollbacks natively.

1.4 Core Terminology You Will See Repeatedly

Term	Meaning
Cluster	A set of machines (nodes) running Kubernetes, managed as a single unit.
Node	A single machine (VM or physical) that runs containerized workloads.
Pod	The smallest deployable unit in Kubernetes wraps one or more containers.
Container image	A packaged, runnable snapshot of an application and its dependencies.
Control Plane	The set of components that make global decisions about the cluster (scheduling, scaling, etc.).
Manifest	A YAML or JSON file describing the desired state of a Kubernetes object.
kubectl	The command-line tool used to interact with a Kubernetes cluster.

2. Kubernetes Architecture

2.1 The Control Plane and Worker Nodes

A Kubernetes cluster has two categories of machines: the control plane, which makes decisions about the cluster, and worker nodes, which actually run application workloads. In a small setup (like minikube) both roles can live on one machine; in production they are almost always separate.

2.2 Control Plane Components

Component	Role
kube-apiserver	The front door to the cluster. Every kubectl command, controller, and component talks to the API server. It validates and processes REST requests and is the only component that talks directly to etcd.
etcd	A distributed, consistent key-value store that holds all cluster data — the single source of truth for cluster state.
kube-scheduler	Watches for newly created Pods with no assigned node and picks the best node for them based on resource needs, constraints, and policies.
kube-controller-manager	Run controller loops (Node controller, Job controller, Deployment controller, etc.) that watch cluster state and drive it toward the desired state.
cloud-controller-manager	Integrates the cluster with a cloud provider's APIs (load balancers, storage volumes, node lifecycle) when running in the cloud.

2.3 Worker Node Components

Component	Role
kubelet	An agent running on every node. It ensures containers described in PodSpecs are running and healthy, and reports node/pod status back to the API server.
kube-proxy	Maintains network rules on each node, enabling communication to Pods from inside and outside the cluster (implements the Service abstraction).

Component	Role
Container runtime	The software that actually runs containers — typically containerd or CRI-O, accessed via the Container Runtime Interface (CRI).

2.4 How a Request Flows Through the Cluster

Understanding this flow demystifies almost everything else in Kubernetes:

1. You run `kubectl apply -f deployment.yaml`. `kubectl` sends the manifest to the `kube-apiserver` as a REST call.
2. The API server validates the request and persists the desired state as an object in `etcd`.
3. The `Deployment controller` (inside `kube-controller-manager`) notices a new `Deployment` and creates a `ReplicaSet`, which in turn creates Pod objects.
4. The scheduler notices unscheduled Pods and assigns each one to a suitable node, updating the Pod object with that node's name.
5. The `kubelet` on the assigned node notices a Pod scheduled to it, pulls the container image, and starts the container via the container runtime.
6. `kube-proxy` updates networking rules so the new Pod is reachable through any `Services` that select it.
7. The `kubelet` continuously reports Pod health back to the API server, which updates the object's status in `etcd`.

2.5 Cluster Add-ons

- `CoreDNS` — provides DNS-based service discovery inside the cluster
- CNI plugin (`Calico`, `Cilium`, `Flannel`, etc.) — implements the pod networking model
- `Metrics Server` — collects resource usage data used by `kubectl top` and autoscalers
- `Ingress Controller` — implements `Ingress` objects to route external HTTP(S) traffic

3. Installing and Setting Up Kubernetes

3.1 Choosing a Learning Environment

Tool	Use Case
minikube	A single-node cluster running in a VM or container on your laptop. Best for beginners and local development.
kind (Kubernetes in Docker)	Runs cluster nodes as Docker containers. Fast, lightweight, great for CI pipelines and multi-node testing.
k3s	A lightweight, certified Kubernetes distribution ideal for edge devices, IoT, and resource-constrained environments.
kubeadm	A tool for bootstrapping a production-style cluster on real or virtual machines you manage yourself.
Managed cloud (EKS/GKE/AKS)	Cloud providers run the control plane for you. Best for production workloads.

3.2 Installing kubectl

kubectl is required regardless of which cluster you use.

```
# macOS (Homebrew)
brew install kubectl

# Linux
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

# Verify installation
kubectl version --client
```

3.3 Setting Up minikube

```
# Install (macOS example)
brew install minikube
```

```
# Start a local cluster
minikube start --driver=docker --cpus=4 --memory=8192

# Check cluster status
minikube status

# Open the Kubernetes dashboard
minikube dashboard

# Stop / delete the cluster
minikube stop
minikube delete
```

3.4 Setting Up kind

```
# Install kind
go install sigs.k8s.io/kind@latest

# Create a cluster
kind create cluster --name dev-cluster

# Create a multi-node cluster from a config file
cat <<EOF > kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF
kind create cluster --name multi-node --config kind-config.yaml

# Delete a cluster
kind delete cluster --name dev-cluster
```

3.5 Bootstrapping a Cluster with kubeadm (Production-Style)

This is a simplified outline of what kubeadm does on real infrastructure — useful for understanding what managed cloud providers automate for you.

```
# On every node: install a container runtime, kubeadm, kubelet, kubectl
sudo apt-get update && sudo apt-get install -y containerd kubeadm kubelet kubectl

# On the control-plane node: initialize the cluster
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# Configure kubectl for the current user
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Install a CNI plugin (example: Flannel)
kubectl apply -f
https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.y
ml

# On each worker node, join the cluster using the command printed by kubeadm init
sudo kubeadm join <control-plane-ip>:6443 --token <token> --discovery-token-ca-cert-hash <hash>
```

3.6 Verifying Your Cluster

```
kubectl cluster-info
kubectl get nodes
kubectl get nodes -o wide
kubectl get pods -A      # all pods, all namespaces
kubectl get componentstatuses
```


4. kubectl Essentials

4.1 Anatomy of a kubectl Command

Most kubectl commands follow the same shape:

```
kubectl <verb> <resource-type> <resource-name> [flags]
```

Example:

```
kubectl get pods my-app-pod -n production -o yaml
```

* In Kubernetes, context usually means the current cluster configuration that kubectl uses.

* Context = Which Kubernetes cluster + which user + which namespace am I currently working with?

* Kubernetes stores contexts in a configuration file, usually: ~/.kube/config

4.2 Context and Configuration

Command	Description
kubectl config view	Show the current kubeconfig (clusters, users, contexts) kubeconfig is k8s configuration file
kubectl config get-contexts	List all available contexts
kubectl config current-context	Show the currently active context
kubectl config use-context <name>	Switch to a different cluster/context
kubectl config set-context --current --namespace=<ns>	Set the default namespace for the current context

4.3 Reading Cluster State

Command	Description
kubectl get <resource>	List resources of a given type
kubectl get all	List common resource types in the current namespace
kubectl get pods -o wide	List pods with extra columns (node, IP, etc.)
kubectl get pods -w	Watch resources for live changes
kubectl describe <resource> <name>	Show detailed information and recent events
kubectl explain <resource>.<field>	Show API documentation for a resource or field
kubectl get pods --show-labels	List pods along with their labels
kubectl get pods -l app=web	Filter resources by label selector

4.4 Creating and Modifying Resources

Command	Description
<code>kubectl apply -f file.yaml</code>	Create or update a resource declaratively from a manifest
<code>kubectl apply -f dir/</code>	Apply every manifest in a directory
<code>kubectl create -f file.yaml</code>	Create a resource imperatively (fails if it already exists)
<code>kubectl delete -f file.yaml</code>	Delete the resource(s) defined in a manifest
<code>kubectl delete pod <name></code>	Delete a specific pod by name
<code>kubectl edit <resource> <name></code>	Open a resource in your editor and apply changes live
<code>kubectl scale deployment <name> --replicas=5</code>	Change the number of replicas
<code>kubectl label pod <name> tier=frontend</code>	Add or update a label on a resource
<code>kubectl annotate pod <name> note="reviewed"</code>	Add or update an annotation

4.5 Debugging and Interacting with Running Pods

Command	Description
<code>kubectl logs <pod></code>	Show logs for a pod's (only) container
<code>kubectl logs <pod> -c <container></code>	Show logs for a specific container in a multi-container pod
<code>kubectl logs -f <pod></code>	Stream (follow) logs in real time
<code>kubectl logs --previous <pod></code>	Show logs from a previous (crashed) container instance
<code>kubectl exec -it <pod> -- /bin/sh</code>	Open an interactive shell inside a running container
<code>kubectl port-forward pod/<name> 8080:80</code>	Forward a local port to a port on the pod
<code>kubectl cp <pod>:/path ./local-path</code>	Copy files between a pod and your local machine
<code>kubectl top pods</code>	Show live CPU/memory usage per pod (requires metrics-server)

Command	Description
<code>kubectl top nodes</code>	Show live CPU/memory usage per node

4.6 Output Formatting

Command	Description
<code>-o wide</code>	Additional columns in table view
<code>-o yaml</code>	Full object as YAML
<code>-o json</code>	Full object as JSON
<code>-o jsonpath='{.status.podIP}'</code>	Extract a specific field using JSONPath
<code>-o name</code>	Print only resource names — useful for scripting

4.7 Dry Runs and Generating YAML Quickly

A very common workflow: generate a manifest skeleton imperatively, then refine it as a YAML file you keep in version control.

```
# Generate a Deployment manifest without creating it
```

```
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > deployment.yaml
```

```
# Generate a Service manifest without creating it
```

```
kubectl expose deployment nginx --port=80 --dry-run=client -o yaml > service.yaml
```

```
# Validate a manifest against the API server without persisting it
```

```
kubectl apply -f deployment.yaml --dry-run=server
```

Best Practice

Use `kubectl` with version-controlled YAML files for anything beyond quick experiments. Imperative commands (`kubectl run`, `kubectl create`) are great for learning and throwaway testing, but declarative manifests are the standard for real environments.

5. Pods, ReplicaSets, and Deployments

5.1 Pods: The Atomic Unit

A Pod represents one instance of a running process in your cluster. It can hold one container (the common case) or several tightly-coupled containers that share network and storage. Pods are ephemeral — they are not meant to be updated in place; when something changes, Kubernetes replaces the Pod with a new one.

apiVersion: v1

kind: Pod

metadata:

name: web-pod

labels:

app: web

spec:

containers:

- name: web

image: nginx:1.27

ports:

- containerPort: 80

resources:

requests:

cpu: "100m"

memory: "128Mi"

limits:

cpu: "250m"

memory: "256Mi"

Spec

Key Takeaways

Kubernetes is a container orchestration platform that coordinates the collaboration of Master Nodes and Worker Nodes. Master Nodes (Control Plane) are responsible for scheduling and deciding where applications run. Worker Nodes provide the infrastructure to actually run the applications. In a single-node cluster, your computer plays the role of Master and Worker Node.

Containers

Containers run applications in isolation with their dependencies, making them highly portable.

Pods

In Kubernetes, pods are the smallest deployable units and encapsulate application containers.

Pod Configuration

Metadata:

Name: Uniquely identifies the pod

Labels: Categorize pods into distinct groups for flexible querying

Runtime Requirements (specified under 'spec'):

Container name

Image source

Port for serving requests

Resource requirements (CPU and memory)

Memory limit and request should be the same

CPU limit should rarely be set (as per Kubernetes best practices)

spec: # Specification/spec a run-time configuration, What should Kubernetes actually run?, what specification for running container in pod

Multi-Container Pods

Pods can run multiple containers, enabling sidecar patterns where auxiliary sidecar containers can communicate with the main application container via localhost.

Port Forwarding in Kubernetes

Port forwarding creates a temporary connection between your local machine and a pod in the cluster. It's primarily used for debugging and testing purposes. The command structure is: `kubectl port-forward <pod-name> <local-port>:<pod-port>`. For example, `kubectl port-forward mypod 8080:80` forwards local port 8080 to port 80 of the pod.

Command	Description
<code>kubectl run web-pod --image=nginx</code>	Create a single pod imperatively
<code>kubectl apply -f pod.yaml</code>	Create a pod from a manifest
<code>kubectl get pods</code>	List pods in the current namespace
<code>kubectl describe pod web-pod</code>	Show detailed status and recent events
<code>kubectl delete pod web-pod</code>	Delete a pod

5.2 Why You Rarely Create Pods Directly

A bare Pod has no self-healing behavior — if its node fails, it is gone. In practice you almost always manage Pods through a higher-level controller that supervises them.

5.3 ReplicaSets

ReplicaSet makes sure the correct number of Pods are running.

A ReplicaSet ensures a specified number of identical Pod replicas are running at all times. If a Pod dies, the ReplicaSet creates a replacement; if there are too many, it deletes the excess.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

Behind the scenes:

- #1. The Deployment Controller creates a ReplicaSet based on this specification.
- #2 A ReplicaSet is a Kubernetes resource that ensures a specified number of pod replicas are running at all times.
- #3. The ReplicaSet Controller watches for ReplicaSet objects and creates pods based on them.
- #4. The ReplicaSet Controller continuously monitors the state of the pods.
- #5 If a pod terminates or fails, the ReplicaSet Controller automatically creates a new one to maintain the desired number of replicas.

Deployment manages ReplicaSets and provides safe application updates.

5.4 Deployments: The Recommended Way to Run Stateless Apps

A Deployment manages ReplicaSets on your behalf and adds rolling updates, rollback history, and pause/resume control. In practice, Deployments are the default way to run stateless workloads.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  labels:
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
```

* A Deployment in Kubernetes allows us to declaratively manage a set of pods.

* The two key specifications in a Deployment are the number of pod replicas, and the pod template (defining how each pod should be constructed). This Deployment declares our desired state: 3 replicas of pods based on the specified template:

* The Deployment Controller manages the overall lifecycle of the Deployment, including updates and rollbacks

while the ReplicaSet Controller handles the day-to-day management of pods. This orchestration happens automatically. As developers, we simply declare our desired state in the Deployment object, and Kubernetes handles the rest, ensuring our application remains running as specified.

ROLLING UPDATES

when we upgrade instances, we do not have to upgrade all of them at once as we just did. This may impact users accessing our applications, so we may want to upgrade them one after the other. And that kind of upgrade is known as Rolling Updates.

ROLLBACK

Suppose one of the upgrades we performed resulted in an unexpected error and we are asked to undo the recent update. We would like to be able to rollBACK the changes that were recently carried out.

Remark : Both Rolling updates are controlled by specifying in the manifest via strategy attributed

```

app: web
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
template:
  metadata:
    labels:
      app: web
  spec:
    containers:
      - name: web
        image: nginx:1.27
        ports:
          - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
            initialDelaySeconds: 5
        livenessProbe:
          httpGet:
            path: /
            port: 80
            initialDelaySeconds: 15

```

strategy: defines how the rolling

type: type of updates performed

maxUnavailable: The maximum number of pods that can be unavailable during the update.

maxSurge: The maximum number of pods that can be scheduled above the desired number of pods.

Notice:
Both attribute takes a value that can be an absolute number (ex: 5) or a percentage of desired pods (ex: 10%). This can not be 0 if MaxUnavailable is 0. Absolute number is calculated from percentage by rounding up. Defaults to 25%.
Example: when this is set to 30%, the new ReplicaSet can be scaled up immediately when the rolling update starts, such that the total number of old and new pods do not exceed 130% of desired pods. Once old pods have been killed, new ReplicaSet can be scaled up further, ensuring that total number of pods running at any time during the update is at most 130% of desired pods.

5.4.1 Deployment Commands

Command	Description
<code>kubectl create deployment web --image=nginx --replicas=3</code>	Create a Deployment imperatively
<code>kubectl apply -f deployment.yaml</code>	Create or update a Deployment declaratively
<code>kubectl get deployments</code>	List Deployments
<code>kubectl rollout status deployment/web-deployment</code>	Watch a rollout until it completes
<code>kubectl set image deployment/web web=nginx:1.28</code>	Trigger a rolling update to a new image
<code>kubectl rollout history deployment/web-deployment</code>	Show revision history

Command	Description
<code>kubectl rollout undo deployment/web-deployment</code>	Roll back to the previous revision
<code>kubectl rollout undo deployment/web --to-revision=2</code>	Roll back to a specific revision
<code>kubectl rollout pause deployment/web-deployment</code>	Pause a rollout (batch multiple changes)
<code>kubectl rollout resume deployment/web-deployment</code>	Resume a paused rollout
<code>kubectl scale deployment/web --replicas=5</code>	Manually scale the Deployment

How Rolling Updates Work

A Deployment creates a new ReplicaSet for the updated Pod template and gradually shifts traffic from the old ReplicaSet to the new one, respecting `maxUnavailable` and `maxSurge`. The old ReplicaSet is kept (scaled to zero) so a rollback is instant.

5.5 Probes: Letting Kubernetes Know Your App Is Healthy

Probe Type	Effect When It Fails
Liveness probe	If it fails, Kubernetes restarts the container. Use it to catch deadlocks.
Readiness probe	If it fails, the Pod is removed from Service endpoints — it stays running but stops receiving traffic.
Startup probe	Delays liveness/readiness checks until a slow-starting app has finished initializing.

Why do we need probes?

Liveness Probe and Readiness Probe are Kubernetes health checks for containers. Imagine your application starts:

```
Pod
├── Container
│   └── Application
```

Just because the container is running does not mean the application is healthy.

For example:

Container: Running ✓

Application:

- Database connection failed x
- Application is stuck x
- Still starting x

Kubernetes probes help detect these situations.

1. Liveness Probe
2. Readiness Probe:
 - Prevent traffic from reaching the pod if its application is 'not ready', i.e. an application could be operational but not ready to serve traffic.
 - Because the application might rely on external services such as database or redis, rabbitmq or other

Liveness

The liveness endpoint returns a 200 status if the application is operational, or a 500 status if it's not. The liveness probe checks this endpoint and considers the app healthy only if it receives a 200 status. Any other response, or no response at all, triggers a container restart.

Readiness

The readiness endpoint verifies if the application has successfully connected to all components necessary for serving traffic. It returns a 200 status only when all required connections and initializations are complete, and a 500 status otherwise. The readiness probe uses this endpoint to determine if a container is ready to accept traffic. If the probe receives anything other than a 200 status, or no response, it keeps the container out of service.

Initial Delay and Period

The initial delay sets how long to wait before the first probe runs, while the period determines the frequency of subsequent probes. For example, if an app takes 20 seconds to start, set the liveness probe's initial delay to at least 20 seconds. This ensures the probe only begins checking after the application has had sufficient time to initialize:

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 25
  periodSeconds: 5
```

For readiness probes, the initial delay is less critical. An unresponsive app during startup correctly indicates it's not yet ready for traffic.

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  periodSeconds: 5
```

Usage Use liveness probes to detect and restart unhealthy containers. Use readiness probes to determine when a container is ready to start accepting traffic. Together, they ensure your application remains healthy and responsive in a Kubernetes environment.

Services and Cluster Networking

6.1 The Problem Services Solve

Pods are **ephemeral** and **get a new IP address every time they are recreated**. A Service provides a **stable virtual IP address** and **DNS name** that automatically load-balances traffic across whichever Pods currently match its label selector.

* A Service in Kubernetes is a resource that provides a stable way to access one or more Pods.

* The main problem a Service solves is that Pods are temporary and their IP addresses can change.

6.2 Service Types

Type	Behavior
ClusterIP (default)	Exposes the Service on an internal IP reachable only from inside the cluster.
NodePort	Exposes the Service on a static port on every node's IP, in addition to a ClusterIP.
LoadBalancer	Provisions an external load balancer from the cloud provider, routing to the Service.
ExternalName	Maps the Service to an external DNS name via a CNAME record, with no proxying.

apiVersion: v1
kind: Service
metadata:
name: web-service
spec:
type: ClusterIP
selector:
app: web
ports:
- protocol: TCP
port: 80
targetPort: 80

Key Takeaways

- The Service primitive in Kubernetes abstracts away network complexities and provides a durable endpoint for accessing pods.

NodePort Service

- Allows external access to the Kubernetes network by exposing a static port on the node (range: 30000-32767).

```
apiVersion: v1
kind: Service
metadata:
  name: grade-submission-portal
spec:
  type: NodePort
  selector:
    app: grade-submission-portal
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080
```

- The external request is initiated on the node's static port (nodePort: 30080).

- The request enters the cluster through the Service's internal port (port: 8080). With NodePort services, this internal port can be any valid port number, as the external nodePort (30080) will be mapped to whatever internal port is specified.

- The service acts as a proxy by directing the request to a matching pod using a label selector.

- The service specifies a target port (targetPort: 8080) that ensures the request reaches the container port on the pod.

Note: The NodePort service is often used when prototyping, rarely in practice.

Command	Description
kubectl expose deployment web --port=80	Create a Service for an existing Deployment
kubectl get svc	List Services

Type	Behavior
<code>kubectl get endpoints web-service</code>	Show which Pod IPs a Service currently routes to
<code>kubectl describe svc web-service</code>	Inspect a Service's selector, ports, and endpoints
<code>kubectl delete svc web-service</code>	Delete a Service

6.3 Service Discovery via DNS

CoreDNS automatically creates a DNS record for every Service. From any Pod in the cluster, you can reach a Service using its name (if in the same namespace) or its fully qualified name.

```
# Same namespace
curl http://web-service

# Fully qualified: <service>.<namespace>.svc.cluster.local
curl http://web-service.production.svc.cluster.local
```

6.4 The Pod Network Model

- Every Pod gets its own IP address; containers within a Pod share that IP and localhost.
- Pods can reach all other Pods' IPs directly, without NAT, regardless of which node they're on.
- This flat network model is implemented by a CNI plugin (Calico, Cilium, Flannel, etc.), not by Kubernetes itself.

6.5 NetworkPolicies

By default, all Pods can reach all other Pods. A NetworkPolicy restricts traffic using label selectors, similar to a firewall for Pod-to-Pod communication. NetworkPolicies require a CNI plugin that supports enforcement (e.g., Calico or Cilium).

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
```

```
    app: backend
policyTypes:
- Ingress
ingress:
- from:
    podSelector:
      matchLabels:
        app: frontend
ports:
- protocol: TCP
  port: 8080
```

7. Configuration: ConfigMaps and Secrets

7.1 Why Separate Configuration From Images

Baking configuration into a container image means rebuilding the image for every environment. ConfigMaps and Secrets let you inject configuration and sensitive values at runtime, so the same image can run in development, staging, and production.

7.2 ConfigMaps

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  MAX_CONNECTIONS: "100"
  app.properties: |
    retries=3
    timeout=30
```

Command	Description
<code>kubectl create configmap app-config --from-literal=LOG_LEVEL=info</code>	Create a ConfigMap from literals
<code>kubectl create configmap app-config --from-file=config.properties</code>	Create a ConfigMap from a file
<code>kubectl get configmaps</code>	List ConfigMaps
<code>kubectl describe configmap app-config</code>	View a ConfigMap's contents

7.2.1 Consuming a ConfigMap

```
spec:
  containers:
    - name: app
      image: myapp:1.0
      envFrom:
        - configMapRef:
            name: app-config
```

```

volumeMounts:
  - name: config-volume
    mountPath: /etc/config
volumes:
  - name: config-volume
    configMap:
      name: app-config

```

7.3 Secrets

Secrets work like ConfigMaps but are intended for sensitive data (passwords, tokens, TLS certificates). Values are base64-encoded (not encrypted by default) — encryption at rest and RBAC access control are what actually protect them, so treat Secrets as sensitive infrastructure, not as a free security guarantee.

```

apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  username: YWRtaW4= # base64 for
'admin'
  password: cGFzc3dvcmQxMjM=

```

Command	Description
<code>kubectl create secret generic db-credentials --from-literal=username=admin --from-literal=password=secret</code>	Create a generic Secret
<code>kubectl create secret tls my-tls --cert=cert.crt --key=cert.key</code>	Create a TLS Secret
<code>kubectl create secret docker-registry regcred --docker-username=u --docker-password=p</code>	Create an image-pull Secret
<code>kubectl get secrets</code>	List Secrets (values hidden)
<code>kubectl get secret db-credentials -o jsonpath='{.data.password}' base64 -d</code>	Decode a Secret value

Security Note

For production, consider an external secrets manager (Vault, AWS Secrets Manager, sealed-secrets) combined with RBAC and encryption at rest, rather than relying on plain Kubernetes Secrets alone.

Key Takeaways

- ConfigMaps in Kubernetes are used to store non-confidential configuration data in key-value pairs, allowing you to decouple configuration from pod specifications and make your applications more portable. They can be consumed by pods as environment variables and updated without rebuilding your application container.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-config
data:
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"
```

Secrets in Kubernetes are similar to ConfigMaps but are specifically designed for sensitive information like passwords, OAuth tokens, and SSH keys. They can be consumed by pods similarly to ConfigMaps. Secrets use the data field, which expects base64 encoded values. This encoding is particularly useful for handling special characters often found in sensitive data:

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  username: YWRtaW4= # base64 encoded 'admin'
  password: dDBwLVZmZy3lzdA== # base64 encoded 't0p-S3cr3t'
```

Important Note: while Secrets are base64 encoded, they are not encrypted. Additional security measures are typically implemented to protect sensitive data in clusters. These can include external secret management systems, or implementing Kubernetes Encryption Providers. However, specific approaches are beyond the scope of this overview as they vary based on organizational needs and security policies.

8. Storage: Volumes, PVs, PVCs, and StorageClasses

8.1 Why Pods Need External Storage

A container's filesystem is ephemeral — it disappears when the container restarts. Volumes give Pods a way to persist or share data beyond a single container's lifecycle.

8.2 Volume Types (Quick Reference)

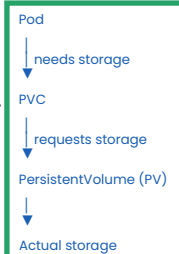
Volume Type	Description
emptyDir	Temporary storage created when a Pod is assigned to a node; deleted when the Pod is removed.
hostPath	Mounts a file or directory from the host node's filesystem. Use sparingly; ties Pods to specific nodes.
configMap / secret	Mounts configuration or secret data as files inside the Pod.
persistentVolumeClaim	The standard way to request durable, provider-backed storage (see below).

8.3 PersistentVolumes and PersistentVolumeClaims

A PersistentVolume (PV) represents an actual piece of storage in the cluster (an EBS volume, NFS share, etc.). A PersistentVolumeClaim (PVC) is a request for storage made by a user or application.

A PVC is how a Pod requests persistent storage from Kubernetes.

Kubernetes binds a PVC to a matching PV, decoupling how storage is provisioned from how it is consumed.



```
# PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
```

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

This specifies how the storage can be accessed.
Common modes are:
ReadWriteOnce (RWO) = Storage can be mounted read/write by one node at a time.
ReadOnlyMany (ROX) = Storage can be mounted read-only by multiple nodes at a time.
ReadWriteMany (RWX) = Storage can be mounted read/write by multiple nodes.
The exact support depends on your storage provider.
resources.requests.storage
requests:
storage: 10Gi
This means: "I need 10 GiB of storage."

```
storageClassName: standard
```

```
spec:
```

```

containers:
  - name: app
    image: myapp:1.0
    volumeMounts:
      - name: data
        mountPath: /var/lib/data
volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data-pvc

```

8.4 StorageClasses and Dynamic Provisioning

A StorageClass defines a class of storage (SSD vs HDD, provider parameters, reclaim policy) and enables dynamic provisioning: instead of an administrator pre-creating PVs, Kubernetes creates one automatically whenever a PVC requests that class.

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer

```

8.5 Storage Commands

Command	Description
<code>kubectl get pv</code>	List PersistentVolumes
<code>kubectl get pvc</code>	List PersistentVolumeClaims
<code>kubectl get storageclass</code>	List available StorageClasses
<code>kubectl describe pvc data-pvc</code>	Inspect binding status and events for a PVC
<code>kubectl delete pvc data-pvc</code>	Delete a claim (behavior depends on reclaim policy)
Access Mode	Meaning

Command	Description
ReadWriteOnce (RWO)	Volume can be mounted read-write by a single node
ReadOnlyMany (ROX)	Volume can be mounted read-only by many nodes
ReadWriteMany (RWX)	Volume can be mounted read-write by many nodes

9. Namespaces, Labels, Selectors, and Annotations

9.1 Namespaces: Dividing a Cluster

A Namespace creates a logical partition within a single cluster — useful for separating teams, environments, or applications, and for applying resource quotas and access controls per group.

```
apiVersion: v1
kind: Namespace
metadata:
  name: staging
```

A Namespace is a logical partition inside a Kubernetes cluster used to organize and isolate resources.

It is a kind of a logical workspace inside the same cluster.

Command	Description
<code>kubectl create namespace staging</code>	Create a namespace
<code>kubectl get namespaces</code>	List all namespaces
<code>kubectl get pods -n staging</code>	List pods in a specific namespace
<code>kubectl get pods --all-namespaces</code>	List pods across all namespaces
<code>kubectl delete namespace staging</code>	Delete a namespace and everything inside it
<code>kubectl config set-context --current --namespace=staging</code>	Set the default namespace for your shell session

Not Everything Is Namespaced

Cluster-wide resources like Nodes, PersistentVolumes, and StorageClasses do not belong to a namespace — they exist at the cluster level.

9.2 Labels: Arbitrary, Queryable Key-Value Tags

Labels are attached to objects and used by selectors to group resources — this is how Services find Pods, how Deployments find their ReplicaSets, and how you organize resources for querying.

```
metadata:
  labels:
    app: web
    tier: frontend
    environment: production
```

Command	Description
---------	-------------

<code>kubectl get pods -l app=web</code>	List pods with a specific label
<code>kubectl get pods -l 'environment in (staging,production)'</code>	List pods matching a set-based selector
<code>kubectl label pod my-pod tier=frontend</code>	Add a label to a running pod
<code>kubectl label pod my-pod tier-</code>	Remove a label from a pod

9.3 Annotations: Non-Identifying Metadata

Annotations also attach key-value data to objects, but are not used for selection — they hold metadata like build info, contact details, or configuration hints for tools (e.g., Ingress controller settings).

```
metadata:
  annotations:
    kubernetes.io/change-cause: "Deployed release 2.4.1"
    nginx.ingress.kubernetes.io/rewrite-target: /
```

9.4 ResourceQuotas and LimitRanges

A ResourceQuota caps total resource consumption within a namespace; a LimitRange sets default and maximum resource requests/limits for individual objects inside it.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: staging-quota
  namespace: staging
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
```

10. Scheduling: Affinity, Taints, and Resource Management

10.1 How the Scheduler Picks a Node

The scheduler filters out nodes that can't run a Pod (insufficient resources, taints, port conflicts), then scores the remaining nodes to find the best fit. You can influence this process with resource requests, node selectors, affinity rules, and taints/tolerations.

10.2 Resource Requests and Limits

Requests tell the scheduler how much CPU/memory a container needs to be placed on a node. Limits cap how much it's allowed to use at runtime.

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

What Happens at the Limit

Exceeding a memory limit gets a container OOMKilled and restarted. Exceeding a CPU limit does not kill the container — it simply gets throttled.

10.3 nodeSelector: The Simple Case

```
spec:
  nodeSelector:
    disktype: ssd
```

10.4 Node Affinity: Flexible Placement Rules

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
```

```

      operator: In
      values: ["ssd"]
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
        preference:
          matchExpressions:
            - key: zone
              operator: In
              values: ["us-east-1a"]

```

10.5 Pod Affinity and Anti-Affinity

Pod affinity places Pods near other Pods (e.g., co-locate a cache with its app for low latency); anti-affinity spreads Pods apart (e.g., ensure replicas land on different nodes for resilience).

```

spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: ["web"]
          topologyKey: "kubernetes.io/hostname"

```

10.6 Taints and Tolerations

A taint on a node repels Pods unless they carry a matching toleration. This is the inverse of affinity: instead of a Pod choosing a node, the node rejects Pods that don't explicitly tolerate it. Common uses include dedicating nodes to specific teams or isolating GPU nodes.

Command	Description
<code>kubectl taint nodes node1 key=value:NoSchedule</code>	Prevent new Pods without a matching toleration from scheduling
<code>kubectl taint nodes node1 key=value:NoSchedule-</code>	Remove a taint from a node
<code>kubectl describe node node1 grep Taints</code>	View a node's current taints
spec:	

tolerations:

- key: "key"
- operator: "Equal"
- value: "value"
- effect: "NoSchedule"

Taint Effect	Behavior
NoSchedule	New Pods without a toleration will not be scheduled; existing Pods are unaffected
PreferNoSchedule	The scheduler tries to avoid placing Pods here, but it isn't guaranteed
NoExecute	New Pods are not scheduled and existing Pods without a toleration are evicted

11. StatefulSets, DaemonSets, Jobs, and CronJobs

11.1 StatefulSets: For Applications That Need Stable Identity

Unlike Deployments, StatefulSets give each Pod a stable, unique network identity and stable storage that persists across rescheduling — essential for databases, message queues, and other stateful systems.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 20Gi
```

Key Takeaways

- Managing stateful applications and their storage requirements is time-consuming and error-prone.

Automatic Storage Orchestration

With Kubernetes, you can fully rely on the control plane to manage storage for your stateful applications:

Automatic provisioning and attachment of storage volumes

Seamless data persistence across pod lifecycle events

Built-in mechanisms for maintaining pod identity and state

Key Components of StatefulSet Storage

1. Persistent Volume Claims (PVCs): Requests for storage resources

2. Volume Claim Templates:

- Define the PVC specification within the StatefulSet

- Automatically generate PVCs for each pod in the StatefulSet

- Example: If a StatefulSet has 3 replicas, 3 PVCs will be dynamically created

3. Persistent Volumes (PVs): Actual storage resources that fulfill PVCs

4. Storage Orchestration:

- Each StatefulSet pod gets its own PVC, generated from the template

- PVC binds to an appropriate PV

- PV is mounted at the specified directory in the stateful container

- Pods are named predictably: postgres-0, postgres-1, postgres-2
- Each Pod gets its own PersistentVolumeClaim that survives rescheduling
- Pods are created and terminated in order, one at a time by default

11.2 DaemonSets: One Pod Per Node

A DaemonSet ensures a copy of a Pod runs on every (or a selected subset of) node — the standard pattern for log collectors, monitoring agents, and CNI/network daemons.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-agent
spec:
  selector:
    matchLabels:
      app: log-agent
  template:
    metadata:
      labels:
        app: log-agent
    spec:
      containers:
        - name: log-agent
          image: fluent-bit:latest
```

11.3 Jobs: Run-to-Completion Tasks

A Job runs one or more Pods until they successfully complete, retrying on failure — ideal for batch processing, migrations, and one-off scripts.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
spec:
  completions: 1
  backoffLimit: 3
  template:
    spec:
      containers:
        - name: migrate
          image: myapp-migrate:1.0
      restartPolicy: Never
```

11.4 CronJobs: Scheduled Jobs

A CronJob creates Jobs on a repeating schedule, using standard cron syntax.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: backup-tool:1.0
          restartPolicy: OnFailure
```

11.5 Commands for Workload Controllers

Command	Description
<code>kubectl get statefulsets</code>	List StatefulSets
<code>kubectl get daemonsets</code>	List DaemonSets
<code>kubectl get jobs</code>	List Jobs
<code>kubectl get cronjobs</code>	List CronJobs
<code>kubectl create job manual-run --from=cronjob/nightly-backup</code>	Manually trigger a CronJob's Job now
<code>kubectl delete job db-migration</code>	Delete a completed or failed Job
<code>kubectl logs job/db-migration</code>	View logs from a Job's Pod

12 . Ingress and Ingress Controllers

12.1 Why Ingress?

A LoadBalancer Service gives you one external IP per Service, which gets expensive and unwieldy with many services. Ingress provides a single entry point that routes HTTP/HTTPS traffic to many Services based on hostname and path — like a reverse proxy configured through Kubernetes objects.

12.2 Ingress Requires an Ingress Controller

An Ingress object is just a set of routing rules; nothing happens until an Ingress controller (NGINX Ingress Controller, Traefik, HAProxy, cloud-native ALB controllers, etc.) is running in the cluster to implement them.

```
# Example: install the NGINX Ingress Controller via its official manifest
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml
```

12.3 A Basic Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80
```

```
- path: /api
pathType: Prefix
backend:
  service:
    name: api-service
    port:
      number: 8080
```

12.4 TLS Termination

```
spec:
  tls:
    - hosts:
        - shop.example.com
      secretName: shop-tls-cert
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80
```

12.5 Ingress Commands

Command	Description
<code>kubectl get ingress</code>	List Ingress objects
<code>kubectl describe ingress web-ingress</code>	Show routing rules and backend status
<code>kubectl get ingressclass</code>	List available Ingress controller classes
Related Tools cert-manager automates TLS certificate issuance (e.g., via Let's Encrypt) and pairs naturally with Ingress. The newer Gateway API is a more expressive successor to Ingress, worth exploring	

Command	Description
once you're comfortable with the basics.	

Helm is a package manager for Kubernetes that simplifies the deployment and management of applications. By using Helm Charts, you can manage complex Kubernetes applications as a single unit, simplifying deployment, upgrades, and rollbacks. This approach provides a more organized and maintainable way to handle Kubernetes resources compared to managing individual loose resources.

Directory Structure

A typical Helm Chart has the following structure:

```
mychart/
├── Chart.yaml      # Contains chart information
├── values.yaml     # Default configuration values
├── templates/      # Directory for template files
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
```

Chart.yaml: Metadata about the chart (name, version, dependencies)

values.yaml: Default configuration values that can be overridden

templates/: Directory containing Kubernetes manifest templates

Common Helm Commands

Install a chart:

```
helm install [RELEASE_NAME] [CHART]
```

Uninstall a release:

```
helm uninstall [RELEASE_NAME]
```

Upgrade a release:

```
helm upgrade [RELEASE_NAME] [CHART]
```

List all releases:

```
helm list
```

View the manifest templates with values applied:

```
helm template [CHART]
```

Package a chart into a versioned archive file:

```
helm package [CHART_PATH]
```

Deployment Workflow

Create or modify chart files in the chart directory.

Use helm template to preview the rendered Kubernetes manifests.

Install or upgrade the release using helm install or helm upgrade.

Monitor the deployment using kubectl commands.

If needed, use helm uninstall to remove the release and all associated resources.

Key Takeaways

Helm serves as a powerful package manager for Kubernetes, simplifying the deployment of complex software like Elasticsearch, MongoDB, MySQL, and Redis.

Process of Deploying Complex Software with Helm

Add Helm Repository: `helm repo add bitnami`
<https://charts.bitnami.com/bitnami>

This adds the Bitnami repository, which hosts many popular software charts.

Update Helm Repositories: `helm repo update`

Ensures you have the latest chart versions available.

Search for Available Charts: `helm search repo bitnami/mongodb`

Find the chart you need and check its available versions.

Research Default Values:

Examine the default values.yaml file in the chart's documentation.

Understand which values you need to modify for your use case.

Create Custom Values File:

Create a file, e.g., my-mongodb-values.yaml, with your custom settings.

Only include values that differ from the defaults.

Install the Chart with Custom Values: `helm install my-mongodb bitnami/mongodb -f my-mongodb-values.yaml`

This command installs MongoDB, applying your custom configuration on top of the default values. Only the fields specified in my-mongodb-values.yaml will override the corresponding default values while all other settings

13. Helm: The Kubernetes Package Manager

13.1 The Problem Helm Solves

Real applications are made of many interrelated manifests — Deployments, Services, ConfigMaps, Ingress, RBAC rules. Helm packages these into a single unit called a chart, with templating so the same chart can be reused across environments by changing a small set of values.

13.2 Installing Helm

```
# macOS
brew install helm

# Linux
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

helm version
```

13.3 Key Concepts

Term	Meaning
Chart	A packaged set of Kubernetes manifest templates plus metadata
Release	A specific deployed instance of a chart, with a name and its own values
Values	Configuration passed into a chart's templates (values.yaml or --set flags)
Repository	A collection of charts published for reuse (like a package registry)

13.4 Using Helm

Command	Description
helm repo add bitnami https://charts.bitnami.com/bitnami	Add a chart repository
helm repo update	Refresh local chart repository indexes
helm search repo postgres	Search added repositories for a chart
helm install my-db bitnami/postgresql	Install a chart as a new release
helm install my-db bitnami/postgresql -f custom-values.yaml	Install with custom configuration

Command	Description
<code>helm list</code>	List installed releases in the current namespace
<code>helm status my-db</code>	Show status and details of a release
<code>helm upgrade my-db bitnami/postgresql --set auth.database=shop</code>	Upgrade a release with new values
<code>helm rollback my-db 1</code>	Roll back a release to a previous revision
<code>helm uninstall my-db</code>	Remove a release and its resources
<code>helm show values bitnami/postgresql</code>	Show all configurable values for a chart

13.5 Creating Your Own Chart

```
helm create my-app
```

```
# Resulting structure:
```

```
# my-app/
```

```
# Chart.yaml      - chart metadata
```

```
# values.yaml      - default configuration values
```

```
# templates/       - Kubernetes manifest templates
```

```
# templates/_helpers.tpl - reusable template snippets
```

```
# Render templates locally without installing (great for debugging)
```

```
helm template my-app ./my-app
```

```
# Install your own chart from a local directory
```

```
helm install my-release ./my-app
```

```
# templates/deployment.yaml (excerpt using Helm templating)
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: {{ .Release.Name }}-web
```

```
spec:
```

```
  replicas: {{ .Values.replicaCount }}
```

```
  template:
```

```
    spec:
```

```
      containers:
```

```
        - name: web
```

```
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

14. RBAC and Cluster Security

14.1 Authentication vs. Authorization

Authentication establishes who is making a request (a user, or a Pod's ServiceAccount). Authorization determines what that identity is allowed to do. Kubernetes handles authorization primarily through Role-Based Access Control (RBAC).

14.2 RBAC Building Blocks

Object	Purpose
Role	A set of permissions (verbs on resources) scoped to a single namespace
ClusterRole	The same idea, but scoped cluster-wide (or reusable across namespaces)
RoleBinding	Grants a Role's permissions to a user, group, or ServiceAccount within a namespace
ClusterRoleBinding	Grants a ClusterRole's permissions cluster-wide

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role
```

```
metadata:
```

```
  namespace: staging
```

```
  name: pod-reader
```

```
rules:
```

```
- apiGroups: [""]
```

```
  resources: ["pods"]
```

```
  verbs: ["get", "list", "watch"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: RoleBinding
```

```
metadata:
```

```
  name: read-pods-binding
```

```
  namespace: staging
```

```
subjects:
```

```
- kind: User
```

```
  name: jane.doe@example.com
```

```
  apiGroup: rbac.authorization.k8s.io
```

Object	Purpose
<pre>roleRef: kind: Role name: pod-reader apiGroup: rbac.authorization.k8s.io</pre>	

14.3 RBAC Commands

Command	Description
<code>kubectl get roles -n staging</code>	List Roles in a namespace
<code>kubectl get clusterroles</code>	List cluster-wide Roles
<code>kubectl get rolebindings -n staging</code>	List RoleBindings in a namespace
<code>kubectl auth can-i create pods --namespace staging</code>	Check whether you can perform an action
<code>kubectl auth can-i create pods --as jane.doe@example.com</code>	Check permissions for another user
<code>kubectl create rolebinding view-binding --clusterrole=view --user=jane.doe@example.com -n staging</code>	Bind a built-in ClusterRole to a user in one namespace

14.4 ServiceAccounts: Identity for Workloads

A ServiceAccount is an identity that Pods use to authenticate to the API server (for example, an application that needs to list its own Pods, or read a ConfigMap). Every namespace has a default ServiceAccount; best practice is to create scoped, purpose-specific ones.

<pre>apiVersion: v1 kind: ServiceAccount metadata: name: app-sa namespace: staging</pre>
<pre>spec: serviceAccountName: app-sa containers: - name: app image: myapp:1.0</pre>

14.5 Additional Security Layers

- Pod Security Standards — enforce baseline, restricted, or privileged security profiles at the namespace level (replacing the deprecated PodSecurityPolicy)
- SecurityContext — control per-Pod/container settings like running as non-root, read-only root filesystem, and dropped Linux capabilities
- NetworkPolicies — restrict Pod-to-Pod traffic (covered in Chapter 6)
- Image scanning and admission controllers — block untrusted or vulnerable images before they run (e.g., via OPA Gatekeeper or Kyverno)

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
  containers:
    - name: app
      image: myapp:1.0
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
```


15. Autoscaling

Autoscaling means Kubernetes can automatically change the number of Pods or the amount of compute resources based on demand.

15.1 Three Layers of Autoscaling

Layer	What It Scales
Horizontal Pod Autoscaler (HPA)	Adds or removes Pod replicas based on <u>observed metrics (CPU, memory, or custom metrics)</u>
Vertical Pod Autoscaler (VPA)	Adjusts a Pod's CPU/memory requests and limits automatically based on <u>usage history</u>
Cluster Autoscaler	Adds or removes worker nodes based on whether Pods are pending due to <u>insufficient capacity</u>

15.2 Horizontal Pod Autoscaler

The HPA requires metrics-server (or a custom metrics pipeline) to be running in the cluster.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

- The Kubernetes Metrics Server is a scalable, lightweight, cluster-wide aggregator of resource usage data. It collects real-time CPU and memory metrics from Kubelets on every node and exposes them via the Kubernetes API server.

- It serves as the core data engine behind built-in Kubernetes autoscaling pipelines, such as the Horizontal Pod Autoscaler (HPA) and Vertical Pod Autoscaler (VPA)

Command	Description
<code>kubectl autoscale deployment web --min=2 --max=10 --cpu-percent=70</code>	Create an HPA imperatively
<code>kubectl get hpa</code>	List HorizontalPodAutoscalers and their current status
<code>kubectl describe hpa web-hpa</code>	Show scaling events and metric history

15.3 Vertical Pod Autoscaler

VPA is a separate add-on (not built into core Kubernetes). It's typically run in "recommendation" mode first, so you can review suggested resource values before allowing it to apply changes automatically — applying changes usually requires restarting the Pod.

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: web-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deployment
  updatePolicy:
    updateMode: "Off" # Off = recommendations only
```

15.4 Cluster Autoscaler

The Cluster Autoscaler watches for Pods that cannot be scheduled due to insufficient resources and requests new nodes from the cloud provider; it also scales down nodes that are underutilized and whose Pods can be safely moved elsewhere. It is configured per cloud provider (EKS, GKE, AKS each have their own setup) rather than through a single generic manifest.

16. Monitoring, Logging, and Observability

16.1 The Three Pillars

Pillar	What It Captures
Metrics	Numeric time-series data (CPU, memory, request rate, error rate) — typically Prometheus
Logs	Text records of events emitted by applications and system components
Traces	End-to-end records of a request's path through multiple services

16.2 Metrics with Prometheus and Grafana

Prometheus scrapes metrics endpoints exposed by applications and cluster components and stores them as time series; Grafana visualizes that data in dashboards. Together they form the most common open-source monitoring stack for Kubernetes.

```
# Common install path: the kube-prometheus-stack Helm chart
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring
--create-namespace
```

16.3 Basic Metrics Commands

Command	Description
<code>kubectl top nodes</code>	Show CPU/memory usage per node
<code>kubectl top pods -A</code>	Show CPU/memory usage per pod, all namespaces
<code>kubectl get --raw /metrics</code>	Query raw API server metrics

16.4 Centralized Logging

Container logs are ephemeral by default — they disappear with the Pod. A logging stack (commonly Fluent Bit or Fluentd shipping to Elasticsearch/Loki, visualized in Kibana/Grafana) collects and retains logs centrally.

Command	Description
<code>kubectl logs <pod></code>	View logs for the current container instance
<code>kubectl logs -f <pod></code>	Stream logs live

Command	Description
<code>kubectll logs --since=1h <pod></code>	View logs from the last hour
<code>kubectll logs --tail=100 <pod></code>	View the last 100 lines
<code>kubectll logs -l app=web --all-containers</code>	View logs from every pod matching a label

16.5 Events: The First Place to Look

Command	Description
<code>kubectll get events</code>	List recent cluster events, most recent last
<code>kubectll get events --sort-by=.metadata.creationTimestamp</code>	Sort events chronologically
<code>kubectll get events -n staging --field-selector involvedObject.name=web-pod</code>	Filter events for a specific object

17. Custom Resources and Operators

17.1 Extending the Kubernetes API

Kubernetes lets you define entirely new object types using a CustomResourceDefinition (CRD). Once registered, instances of that type behave like any built-in object — you can `kubectl get/apply/describe` them, and they're stored in etcd just like Pods or Deployments.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.example.com
spec:
  group: example.com
  scope: Namespaced
  names:
    plural: databases
    singular: database
    kind: Database
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                engine:
                  type: string
                version:
                  type: string
```

17.2 The Operator Pattern

A CRD alone just stores data — nothing happens automatically. An Operator pairs a CRD with a custom controller (a program running in the cluster) that watches instances of that resource and takes real action to reconcile actual state with desired state, encoding operational knowledge that would otherwise live in a human runbook.

- Example: a PostgresCluster CRD paired with an operator that provisions StatefulSets, handles failover, and manages backups automatically
- Popular frameworks for building operators: Kubebuilder and the Operator SDK
- Many production databases and complex systems ship official operators (e.g., the Prometheus Operator used by kube-prometheus-stack)

17.3 Working with Custom Resources

Command	Description
<code>kubectl get crds</code>	List all CustomResourceDefinitions in the cluster
<code>kubectl get databases</code>	List instances of a custom resource (once its CRD exists)
<code>kubectl describe database my-db</code>	Show details of a custom resource instance
<code>kubectl explain database.spec</code>	Show the schema for a custom resource, if defined

18. Troubleshooting and Debugging

18.1 A Systematic Approach

8. Check the Pod's status: `kubectl get pods` — is it Pending, CrashLoopBackOff, ImagePullBackOff, Running?
9. Describe the Pod for events: `kubectl describe pod <name>` — the Events section at the bottom usually tells you exactly what went wrong.
10. Check logs: `kubectl logs <name>` (and `--previous` if it has restarted).
11. Check whether the Pod is Ready, not just Running — a failing readiness probe won't show as a crash, but the Pod won't receive traffic.
12. If networking is the issue, verify Services, Endpoints, and NetworkPolicies.
13. If it's cluster-wide, check node health and the control plane components.

18.2 Common Pod States and What They Mean

State	Likely Cause
Pending	Not yet scheduled — often insufficient resources, unmet affinity/taint rules, or no matching node
ImagePullBackOff / ErrImagePull	The image name is wrong, private and lacking pull credentials, or the registry is unreachable
CrashLoopBackOff	The container starts and exits repeatedly — check logs for the actual error
OOMKilled	The container exceeded its memory limit and was killed
Terminating (stuck)	Often caused by a finalizer waiting on an external resource, or a slow graceful shutdown

18.3 Debugging Commands

Command	Description
<code>kubectl describe pod <name></code>	Full status, container states, and recent events
<code>kubectl logs <name> --previous</code>	Logs from the container instance before the last restart
<code>kubectl exec -it <name> -- sh</code>	Shell into a running container to inspect it directly
<code>kubectl debug <name> -it --image=busybox</code>	Attach an ephemeral debug container to a running Pod
<code>kubectl get events --sort-by=.lastTimestamp</code>	Chronological cluster-wide event feed

Command	Description
<code>kubectl get pods -o wide</code>	See which node a Pod landed on and its IP
<code>kubectl describe node <name></code>	Check node capacity, allocatable resources, taints, and conditions
<code>kubectl get pods --field-selector=status.phase=Pending</code>	List only Pending pods
<code>kubectl rollout status deployment/<name></code>	Check whether a rollout is stuck

18.4 Networking-Specific Debugging

```
# Run a temporary debug pod with common network tools
kubectl run tmp-shell --rm -it --image=nicolaka/netshoot -- /bin/bash

# From inside: test DNS resolution
nslookup web-service.default.svc.cluster.local

# Test connectivity to a Service
curl -v http://web-service

# Check kube-proxy is functioning (from a node)
sudo iptables -L -t nat | grep web-service
```

18.5 Control Plane Health

Command	Description
<code>kubectl get componentstatuses</code>	Basic health of control plane components (deprecated on newer clusters; check pods below instead)
<code>kubectl get pods -n kube-system</code>	View control plane component pods on self-managed clusters
<code>kubectl logs -n kube-system kube-apiserver-<node></code>	Check API server logs on a kubeadm cluster

19. Production Best Practices

19.1 Workload Configuration

- Always set resource requests and limits — unbounded Pods can starve their neighbors
- Define liveness and readiness probes for every long-running container
- Run multiple replicas and spread them with pod anti-affinity across nodes/zones
- Set a PodDisruptionBudget so voluntary disruptions (node drains, upgrades) don't take down all replicas at once
- Prefer immutable, versioned image tags over :latest for reproducible deployments

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web
```

19.2 Security

- Apply the principle of least privilege with RBAC — avoid broad ClusterRoleBindings for humans and workloads alike
- Run containers as non-root with a read-only root filesystem where possible
- Scan images for vulnerabilities before they reach the cluster
- Use NetworkPolicies to restrict traffic to only what's needed
- Rotate and externally manage Secrets rather than storing long-lived credentials in plain Secret objects

19.3 Configuration Management

- Keep all manifests in version control; treat the cluster state as generated from Git (GitOps)
- Use Helm or Kustomize to manage environment-specific differences instead of hand-editing YAML per environment
- Avoid manual kubectl edit changes in production — they drift from source control and get silently overwritten by the next apply

19.4 Reliability

- Use rolling updates with sensible maxUnavailable / maxSurge values, and monitor rollout status in CI/CD
- Set up the Horizontal Pod Autoscaler and Cluster Autoscaler together for workloads with variable demand
- Regularly test backups and disaster recovery procedures for stateful workloads, not just take them
- Run resource quotas per namespace to prevent one team's workload from starving another's

19.5 Observability

- Ship logs and metrics off-cluster — a dying node takes its local logs with it
- Alert on symptoms (error rate, latency, saturation) rather than only on low-level infrastructure metrics
- Use kubectl events and structured application logs together; neither alone tells the whole story

19.6 A Pre-Production Checklist

Area	What to Verify
Resources	Requests/limits set; PodDisruptionBudget defined
Health checks	Liveness, readiness, and (if needed) startup probes configured
Security	Non-root user, RBAC scoped, Secrets sourced securely
Scaling	HPA configured; replica count matches expected load and zone spread
Observability	Metrics exposed; logs shipped centrally; alerts defined
Config management	Manifests in Git; no untracked kubectl edit changes
Rollback plan	Rollout history verified; rollback tested at least once

20. Complete kubectl Command Cheat Sheet

20.1 Cluster and Context

Command	Description
<code>kubectl cluster-info</code>	Show cluster endpoint information
<code>kubectl version</code>	Show client and server versions
<code>kubectl config get-contexts</code>	List available contexts
<code>kubectl config use-context <name></code>	Switch active context
<code>kubectl api-resources</code>	List all resource types the API server supports

20.2 Viewing and Finding Resources

Command	Description
<code>kubectl get all -A</code>	List common resources across all namespaces
<code>kubectl get pods -o wide</code>	List pods with node/IP columns
<code>kubectl describe <type> <name></code>	Detailed info and events for one resource
<code>kubectl get pods --sort-by=.metadata.creationTimestamp</code>	Sort pods by age
<code>kubectl get pods -l app=web,tier=frontend</code>	Filter by multiple labels

20.3 Creating, Updating, Deleting

Command	Description
<code>kubectl apply -f file.yaml</code>	Create/update declaratively
<code>kubectl delete -f file.yaml</code>	Delete resources defined in a file
<code>kubectl replace -f file.yaml --force</code>	Delete and recreate a resource
<code>kubectl patch deployment web -p '{"spec":{"replicas":5}}'</code>	Apply a partial (JSON merge patch) update
<code>kubectl scale deployment web --replicas=5</code>	Change replica count

20.4 Deployments and Rollouts

Command	Description
<code>kubectl rollout status deployment/web</code>	Watch rollout progress
<code>kubectl rollout history deployment/web</code>	Show revision history
<code>kubectl rollout undo deployment/web</code>	Roll back to previous revision
<code>kubectl set image deployment/web web=nginx:1.28</code>	Update container image

20.5 Debugging

Command	Description
<code>kubectl logs -f <pod></code>	Stream logs
<code>kubectl exec -it <pod> -- sh</code>	Shell into a container
<code>kubectl describe pod <name></code>	Events and status detail
<code>kubectl top pods / top nodes</code>	Live resource usage
<code>kubectl get events --sort-by=.lastTimestamp</code>	Recent cluster events

20.6 Namespaces, Labels, RBAC

Command	Description
<code>kubectl create namespace <name></code>	Create a namespace
<code>kubectl label pod <name> key=value</code>	Add a label
<code>kubectl auth can-i <verb> <resource></code>	Check your own permissions
<code>kubectl get rolebindings,clusterrolebindings -A</code>	Audit RBAC bindings cluster-wide

20.7 Storage and Config

Command	Description
<code>kubectl get pv,pvc</code>	List volumes and claims
<code>kubectl create configmap <name> --from-file=<path></code>	Create a ConfigMap from a file
<code>kubectl create secret generic <name> --from-literal=key=value</code>	Create a Secret from a literal